Contents

1. What Is a Computer Programming Language?.....0
2. Types of Programming Languages.....0
i. Machine Language (Low Level Language).....0
ii. Assembly Language (Middle Level Language).....0
iii. High Level Language.....1
3. Elements of Programming Languages.....1
4. Compiler and Interpreter.....2
Comparison Between Compiler and Interpreter.....2

1. What Is a Computer Programming Language?

- Computer programming languages let people give instructions to a computer.
- Writing these instructions is called computer programming.
- A program is a set of instructions for the computer to do a specific task.

2. Types of Programming Languages

i. Machine Language (Low Level Language)

- Uses binary numbers: 1s and 0s.
- Written for the computer's hardware.
- Directly controlled by the computer's CPU.

ii. Assembly Language (Middle Level Language)

- Uses symbols like letters, numbers, and special characters.
- Uses words called mnemonics instead of binary code.

- Needs a translator called an assembler to change mnemonics into machine language.

iii. High Level Language

- Easy for people to understand.
- Similar to human languages like English.
- Uses words and grammar rules to create instructions.
- Needs to be converted into machine language by a compiler or interpreter for the computer to understand.
- Examples: FORTRAN, C, C++, Java, Python, .NET.

3. Elements of Programming Languages

i. **Programming Environment:** The tools and software used to write programs.

ii. **Basic Syntax:** The rules for writing correct instructions.

iii. **Data Types:** The kinds of data (like numbers, text) used in programs.

iv. **Variables:** Names used to store data.

v. **Keywords:** Predefined words used in the language.

vi. **Basic Operators:** Symbols for operations like addition and subtraction.

vii. **Decision Making:** Instructions for making choices (like if-else statements).

viii. **Loops:** Instructions for repeating actions.

Credence IT Professional Training Institute

ix. **Numbers:** Using numbers in programs.

x. **Characters:** Using single letters and symbols.

xi. **Strings:** Using sequences of characters (like words).

xii. **Functions:** Groups of instructions that perform specific tasks.

xiii. **File I/O:** Reading from and writing to files.

4. Compiler and Interpreter

i. We write computer programs using a high-level language that humans understand. This is called source code.

ii. Computers do not understand high-level language; they only understand machine code, written in 0s and 1s.

iii. To change source code into machine code, we use either a compiler or an interpreter.

iv. Both compilers and interpreters convert high-level language into machine code, but they work differently.

Comparison Between Compiler and Interpreter

Point of Comparison	Compiler	Interpreter
Process	Converts the entire program at once	Converts the program line by line
Execution	Executes the program after converting it	Executes each line as it is converted
Speed	Generally, faster once compiled	Generally slower because it converts line by line
Errors	Shows all errors after compiling the whole program	Shows errors one at a time, as each line is executed

Output	Creates a separate machine code file	Does not create a separate file; runs directly
Usage	Used for programs where speed is important	Used for scripting and situations where quick testing is needed
Examples	C, C++ (languages that use compilers)	Python, JavaScript (languages that use interpreters)

Credence IT Professional Training Institute